Tournament Winner

AlgoExpert · Easy · Arrays

The problem

- Competitions are [home, away] pairs, results[i]==1 means home won.
- A win is worth 3 points; return the team with most points (guaranteed unique).

Intuition

- Every match has exactly one winner worth three points, so a running score table is all we need.

Brute force vs optimal

- Brute force: recount every team's points at the end — still O(n)
- Simplest optimal: one hash map + running leader
- Avoid a second pass by updating the leader on the fly

Approach

- 1. Keep a hash map of team to points
- 2. For each match, credit the winner
- 3. Track the leader as scores change
- 4. Return the best team

Worked example

home	away	result	winner
HTML	C#	0	C#

team	points
C#	3

C# wins (+3) → leader: C#

Worked example

home	away	result	winner
C#	Python	0	Python

team	points
C#	3
Python	3

Python wins (+3) → leader: C#

Worked example

home	away	result	winner
Python	HTML	1	Python

team	points
C#	3
Python	6

Python wins (+3) → leader: Python

Java solution

```java
public class TournamentWinner {
    public static String tournamentWinner(List<List<String>> competitions, List<Integer> results) {
        Map<String, Integer> score = new HashMap<>();
        String best = "";
        score.put(best, 0);
        for (int i = 0; i < competitions.size(); i++) {
            String winner = results.get(i) == 1 ? competitions.get(i).get(0) : competitions.get(i).get(1);
            int s = score.merge(winner, 3, Integer::sum);
            if (s > score.get(best)) best = winner;
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(k) teams

Edge cases & pitfalls

- Result = 1 means the HOME team won
- Ties can't happen (problem guarantees one winner)
- Initialise the leader with an empty team at 0 points

Interview tips & follow-ups

- Clarify the meaning of result before coding
- Follow-up: handle ties (e.g. alphabetical)
- Mention O(k) space for k teams

Recap

- Every match has exactly one winner worth three points, so a running score table is all we need.
- Complexity: O(n)